PRACTICAL CODING CHALLENGE

Build a Support Desk System

A mini customer-support platform for managing tickets, responses, and service performance

TIME LIMIT	ASSESSMENT	SUBMISSION
3 hours	Full-stack	Source code + README

1. Objective

Build a working web application that a small customer-support team can use to receive, organise, assign, and resolve customer issues. The application should demonstrate sound full-stack fundamentals, clear workflow logic, reliable data handling, and a usable interface.

2. Functional Requirements

A. Ticket Management

- Create a ticket with a subject, customer name or email, description, priority, and status.
- Display tickets in a list or table with subject, customer, status, priority, assignee, and last-updated time.
- Open a ticket detail view and edit its supported fields.
- Support these statuses: Open, In Progress, Waiting on Customer, and Resolved.
- Support these priorities: Low, Medium, High, and Urgent.
- Allow a ticket to be assigned to a support agent.

B. Replies and Internal Notes

- Allow an agent to add a public reply to a ticket.
- Allow an agent to add an internal note that is clearly distinguished from a public reply.
- Display replies and notes in chronological order in the ticket detail view.
- Store each entry with its content, timestamp, and author name or identifier.

C. Search, Filters, and Sorting

- Search tickets by subject or customer name/email.
- Filter by status, priority, and assignee.
- Sort by most recently updated or by priority.
- Allow multiple filters to be applied together.

D. Activity History

- Record important ticket events, such as creation, assignment changes, status changes, priority changes, and replies/notes.
- Show the event history on the ticket detail page in chronological order.

E. Dashboard

- Show total tickets, open tickets, tickets currently in progress, and resolved tickets.
- Show the number of urgent tickets.
- Show overdue tickets using the rule defined below.
- Dashboard counts must reflect the actual saved ticket data, not hardcoded numbers.

3. Business Rules

- **Required fields:** A ticket must have a subject, customer identifier (name or email), and description. Validate these on both the client and server where applicable.
- **Overdue definition:** A ticket is overdue when it is not Resolved and its created time is more than 24 hours in the past. Display overdue tickets clearly.
- **Resolution:** When a ticket is marked Resolved, it should no longer count as open, in progress, or overdue.
- **History:** Changes to status, priority, or assignee must create an activity-history entry.
- **Persistence:** Tickets, replies/notes, and activity history must remain available after the application is restarted.
- **Data integrity:** Avoid orphaned replies or history entries if a ticket is deleted. You may choose to disallow deletion or implement a consistent deletion policy.

4. Technical Expectations

- Use a frontend, backend API, and persistent data store. Choose a stack you can work with confidently.
- Provide sensible API endpoints for listing, creating, viewing, and updating tickets, and for adding replies/notes.
- Return appropriate success and error responses from the API.
- Validate input and handle missing records and invalid requests gracefully.
- Keep the code reasonably organised and avoid placing all logic in one large file.
- Include setup and run instructions in a README, including any environment variables or database setup required.

5. UI / UX Expectations

- Build a clean, understandable interface that works on a typical laptop screen.
- Make status and priority easy to identify visually.
- Provide useful empty states, validation feedback, and error messages.
- Show loading feedback where requests take time.
- Keep public replies and internal notes visually distinct to reduce the risk of confusion.

6. Optional Stretch Goals

Complete the core requirements first. Stretch features are optional and should not replace incomplete core functionality.

- Basic login using a simple local/demo account.
- Role distinction between Admin and Agent, with a basic permission rule.
- Pagination for the ticket list.
- A configurable response-time target (SLA) and a simple SLA-breach indicator.
- Unit tests for important business rules or API endpoints.

7. Rules and Constraints

- You may use any programming language, framework, or database you are comfortable with.
- AI tools and online documentation are allowed. You are responsible for understanding, verifying, and debugging the code you submit.
- Work independently. Do not ask another person to implement or debug the task for you, and do not share code with other candidates.
- You may ask the interviewer clarifying questions. State assumptions when a requirement is unclear.

- Prioritise a working end-to-end core flow over optional features or visual polish.
- Do not include real customer data, credentials, API keys, or other secrets in your submission.

8. Submission Requirements

- Complete project source code.
- README with setup/run steps and any demo credentials, if applicable.
- Database schema or migration/setup instructions.
- A short note listing what works, what is incomplete, and any known issues.
- A brief explanation of the architecture and how AI tools were used, if used.

9. What We Will Evaluate

Area	What we look for
Functional correctness	Core ticket workflows work end to end.
Backend and data modelling	Clear APIs, appropriate data relationships, persistence, validation.
Problem-solving	Logical debugging, sensible assumptions, handling of edge cases.
Frontend and usability	Clear layout, understandable states, responsive behaviour.
Code quality	Readable structure, separation of concerns, maintainability.
Testing and reliability	Checks critical flows and handles failures.
AI usage and ownership	Verifies generated output and can explain submitted code.

At the end of the assessment, be prepared to walk us through your implementation and explain your key decisions.